Microscopic Traffic Simulation

Special topics on model calibration

Alessandro Scalese

March 13, 2026

Contents

To do:

- **p. 5**: REFINE AFTER WRITING
- **p. 7**: insert figure: proportion of sensors with stat signif res
- **p. 7**: insert figure: average counts for excluded sensors

1. Introduction

Traffic simulation models are invaluable tools in traffic planning and management. Traffic forecasts are one of the foremost information tools used by decision makers in transportation matters such as transportation policies and infrastructure investments, with significant impacts on communities and society as a whole.

In the early 2000s, Flyvbjerg et al (INSERT REFS) extensively analyzed the economic and traffic impacts of infrastructure projects undertaken in the previous decades. Among their findings, they highlighted that more than 50% of all projects reported differences in the forecasted and actual traffic demand above 20%, and that over 90% of the projects incurred in some measure of cost escalation, meaning the final cost was higher than the initial estimates. To top this off, they also underline that these — do not demonstrate — trends in their analysis period (ranging from the 1930s to the end of the 20th century).

This alone already does not paint a — landscape: inaccurate models can lead to possibly wrong or at least not optimal choices in transportation policies and infrastructure design, whose cost is more often than not underestimated, leading to — of public money.

One of the reasons behind the inaccuracy of traffic forecasts is the complexity of traffic modeling itself: a complete traffic model must account for several — components, from the microscopic variables that influence driving behavior, to the drivers of route choice and pathing, all the way up to the definition of the overarching traffic flows. Each of these subcomponents can be modelled in different ways, with different complexities and parameters, and each layer introduces errors and indirections which contribute to the inaccuracy of the final model. Furthermore, the lack of accurate data presents another challenge: the OD trips, which define the flows across the network zones, are not known a priori, but either have to be modeled from socio-economic and topological variables (as done in the four-step model for traffic assignment INSERT REFS) or estimated from traffic measurements (data-driven approach). These measurements are taken from traffic sensors such as induction loops or cameras, and their coverage of city networks is often sparse and uneven, leading to an undetermined optimization problem.

In this context, calibration can then be defined as the systematic adjustment of the model's parameters to minimize the discrepancy between real world measurements and the model's predictions. In transport modeling, these parameters are generally categorized into two groups: supply and demand. Supply parameters describe the physical and operational characteristics of the transportation system, including network geometry, speed limits, signal timings, as well as the microscopic driving behavior parameters that dictate how vehicles interact with the infrastructure (e.g. the lane-changing logic parameters). Demand parameters instead define the general population behavior - i.e. the location and amount of trips, which are typically represented in the OD matrix.

In our case, we are going to use a microscopic traffic simulator, SUMO (INSERT REFS), as our model, using its default driving behavior and transport supply parameters. This leaves the demand parameters - the OD matrix - to be calibrated.

As the simulator acts as a "black-box" function, meaning we can only control the inputs (the OD matrix) and observe the outputs (the sensors measurements), but we do not have an analytical form that describes the relationship between them, we are precluded from using common gradient descent optimization algorithms due to computational limitations. In fact, estimating the gradient using (for example) a Finite Differences approach would require us to compute $2N$ function evaluations (i.e. simulations), where N is the number of parameters, which quickly becomes infeasible as the network grows (as both the number of parameters and the computational requirements of the simulations typically grow with the network's size). Therefore, in transport model calbration, the optimization algorithm choice often falls onto the Simultaneous Perturbation Stochastic Approximation algorithm, originally developed by Spall et al (INSERT REFS), which only requires 2 function evaluations to estimate the local gradient.

This algorithm and one of its variants will be used in the first part of this study, which focuses on the calibration of a traffic model for the city of Aachen. The second part of the study focuses instead on some dynamic applications such as Public Transport control and prioritization.

Indeed, this also showcases one of the unique advantages of SUMO: the ability to use its TraCI (Traffic Control Interface) API for fine-grained control over the simulation state, which allows us to implement real-time measurements and adaptive control strategies.

To do: REFINE AFTER WRITING In the following sections, —: The following chapters are organized as follows: Section 2 describes the Aachen network and the statistical determination of simulation replications; Section 3 details the SPSA-based calibration methodology and results; finally, Section 4 presents the TraCI-based implementation of dynamic Public Transport strategies.

2. Model Calibration

We are tasked with calibrating the origin-demand matrix for the morning peak-hour period (8-9am) for a medium sized network from the city of Aachen, in Germany. The network has been divided in twenty-three TAZes (Traffic Assignment Zones), which define the origin and destination of all trips on the network. This subdivision results in a total of 529 calibrable parameters (OD pairs). The simulation data will be collected by 599 synthetic loop detectors (around 15% coverage), which record counts, speeds and densities on a variety of links across the network.

2.1. Task 1: Evaluating simulation replications

2.1.1. The white noise hypothesis

Our simulator of choice (SUMO with mesoscopic simulations enabled) is inherently stochastic (noisy): several components of the model, which are designed to replicate human driving behavior (lane-changing decision making, speed inconsistencies), introduce various levels of randomicity which have to be accounted for when analysing the model's output. As a consequence of this stochasticity, the same input parameters (OD flows) can lead to different results, meaning a single simulation run is not representative of the average traffic state across the network, but is one of the many possible realizations of the traffic process. To account for this variability, we need to adopt a "white noise" hypothesis, meaning that we assume that the variability in simulation outputs (meaning the error between them and the true values) follows a normal distrbution with zero mean and no systematic bias.

Mathematically, we can express this as:

$$Y_i = \overline{Y} + e \tag{1}$$

where:
$\boldsymbol{Y_i}$ are the outputs of a single simulation run
$\boldsymbol{\overline{Y}}$ is the true expected value of the system
$\boldsymbol{e_i}$ represents the stochastic noise

Under the white noise assumption, e is assumed to be indipenently and identically distributed, justifying the use of the sample mean across multiple simulation replications as a reasonable estimator of the network state.

2.1.2. Evaluating the required replications number

In order to determine the number of samples required to get statistically significant outputs with a 95% confidence interval and a 10% tolerance for each link, we are going to use a collection of data from 100 previous simulation runs. We can compute this number by applying the formula:

$$n = \left[\frac{2t_{(\frac{a}{2}, n-1)} s}{W} \right]^2 \tag{2}$$

where:
n minmum number of required runs

$\mathbf{s}$ standard deviation
$\mathbf{W}$ desired tolerance width
$\boldsymbol{\alpha}$ significance level
$\boldsymbol{t_{(\frac{\alpha}{2},n-1)}}$ student's t statistic

This computation needs to be performed for each sensor, across the range of simulations we want to investigate. A sensor is considered to give statistically significant results if the resulting required number of simulation replications, n, is lower than or equal to the number of simulations for which the test was conducted.

We decided to test the statistical significance of the sensors for up to 30 smulation runs to evaluate the trade-off between significance of the results and processing time. In **To do:** insert figure: proportion of sensors with stat signif res, it can be seen that, for 15 simulation runs, over 90% of the sensors provide statistically significant outputs. Further increasing the number of simulations does not significantly increase the percentage of —, at least not in a way that justifies the increased computational effort.

Furthermore, we also looked into the characteristics of the outputs of the sensors that did not meet the statistical significance requirement for 15 simulation runs: as shown in **To do:** insert figure: average counts for excluded sensors, the majority of the sensors not yielding statistically significant results are characterized by extremely low traffic volumes, which are inherently more sensitive to stochastic fluctuations.

Therefore, we chose to set the number of simulation replications to 15, and to exclude non-compliant sensors from the subsequent calibration process, in order to focus on the sensors with the highest signal-to-noise ratio.

2.2. Task 2: Explolratioin in GoF function and input space

The next task requires us to explore the viability and applicability of multiple goodness of fit functions to our problem, and to analyze and explore the input space to find a more solution (historic, a priori parameters). This initial solution will then be used as input for our calibration / optimization process. The optimization process uses the SPSA (Simultaneous Perturbation Stochastic Approximation) by Spall etal REF to calibrate the input OD matrix. The algorithm's hyperparameters are optimized themselves using an automatic search in the parameter space, powered by the Optuna package in python, which offers a plug-in system to explore it via a Parzen Tree (bayesian optimization etc). We added a slight change to the basic spsa approach: we used common random number generation to run simulation for both the plus and minus perturbations with the same seeds, which should in theory at least help isolate the effects of the perturbations from the simulator's stochasticity. This proved (empirically) to speed up the algorithm convergence.

After this initial optimization approach, we also developed following REF a PC-SPSA algorithm, which leverages Principal Component Analysis of the historical data to reduce the problem space (in our case, from 530 parameters to around 30), which speeds up optimization and also yields better results. The tuning of the hyperparameters was, again, done using optuna in an automatic search approach.

3. Dynamic applications

The implementation of dynamic dwell times more closely reflects reality compared to fixed stop times. in reality, dwell times can vary significantly between stops, due to varying number of people boarding / alighting the bus, and also due to some random human behavior (think for example about people running for the bus which then waits for them, or people almost missing their stop that request the driver to open the door for them).

However, dynamic dwell times also mean that the total travel time of the bus can vary significantly, making timetables inaccurate: this is one of the many reasons behind the — of digital timetables at modern bus stops, which can show a more accurate estimate of the bus ETA compared to fixed timetables which can only take into consideration an average of the historical data. On the environmental side, at least in our simulation, dynamic dwell times lead to a general reduction of the idle time of the bus, therefore causing a reduction in fuel consumption (and so of the emission of pollutants and co2). So: travel time generally down, reliability up

Request stops aim to reduce travel times (and idle times) even further as useless stops are skipped directly (while in the ddt scenarios we still stopped for around 15 seconds at each stop, even if nobody needed to board or alight). Should I talk about the technical difficulties of the implementation? E.g. understanding when to compute the number of people alighting, boarding etc

In general, time travel reduction due to sometimes skipping one of the stops -> the one with fewer requests. This should also lead to a reduction of fuel consumption and emisisons compared to ddt -> understand why it is not shown in the graphs

What I need to do is run the simulation maybe like 4-5 times and then average the results Violin plots should be perfect for this

Public transport prioritization: application to a single junction in the whole network -> if this has good results, think about what would happen with multiple implementations across the network.

Analyze the base program to understand the available phases and how we can play around with them:

- phase bus
- phase “major” -> should probably call sidestream or something
- phase “major-left” -> people that need to turn
- interstage

We place a detector far enough (considering that there is a bus stop before the tl -> bad design) and we try to account for its stop time (V2I hypothesis). After the bus decides if it is going to stop or not, than we apply the logic:

- if phase bus
  - if the bus makes it before the end of the max green, do nothing
  - otherwise,
    - if we have enough time to run a quick cycle (min green times etc) -> do that
    - otherwise, extend green time

- otherwise,
  - if phase is not major
    - wait
  - otherwise,
    - try to extend the major phase as long as possible to avoid disrupting the flow and only change when the bus is approaching 8allowing for a bit of time to let the queue dissipate

Instead of a simple “green time compensation” mechanism, we are going to implement a system that brings the tl back in sync with the adjacent one -> as we found empirically that prioritization would cause spillover and queues there due to people unable to occupy the junction in time. This is done by extending the major phase time and reducing the duration of the bus phase to bring them back in sync.

Very good results: significant reduction in travel times already with just one prioritized traffic light. Good reason to try and implement this in other junctions and for other bus lines as well, good pilot project.

Bibliography